



Nombre: _____ Carnet: _____ Nota: _____

Diseña y escribe una solución para cada uno de los siguientes problemas. Usa Python. (5 puntos cada uno)

1. Escribe un programa que gestione una lista de damnificados (D). El programa debe almacenar los siguientes datos de cada damnificado: nombre, apellido, edad, si perdió su casa y si perdió familiares.

El programa debe contener un procedimiento que almacene los datos de los damnificados en una lista. Luego, debe incluir una función que calcule la cantidad de dinero que recibirá cada damnificado (uno a la vez) en base a los siguientes criterios:

- La cantidad base es de \$1000.
- Si el damnificado es menor de edad, se agrega un 50% adicional a la cantidad total.
- Si el damnificado perdió su casa, se agrega a la cantidad el doble de la cantidad base.
- Si el damnificado perdió familiares, se agrega a la cantidad base por cada familiar perdido.

Por ejemplo, un damnificado que recibe \$1000, perdió su casa (\$2000 adicionales) y perdió 3 familiares (\$3000 adicionales), recibiría un total de \$6000. Si el damnificado es menor de edad, la cantidad total se incrementa en un 50% adicional, resultando en un total de \$9000.

Al finalizar el programa, se debe imprimir el nombre y apellido de cada damnificado, junto con la cantidad de dinero que recibirá.



2. Escribe un programa que permita a los usuarios llevar un registro de sus sesiones de entrenamiento deportivo. El programa debe cumplir con los siguientes requisitos:
- Registro de sesiones: Permitir a los usuarios ingresar los detalles de una sesión de entrenamiento, incluyendo:
 - Fecha de la sesión (por ejemplo, "2024-04-12").
 - Tipo de ejercicio realizado (por ejemplo, "correr", "nadar", "levantar pesas").
 - Duración del ejercicio en minutos.
 - Almacenamiento de datos: Los datos de las sesiones de entrenamiento deben almacenarse en una lista.
 - Reporte de entrenamientos: Permitir a los usuarios ver un resumen de todas sus sesiones de entrenamiento registradas, incluyendo:
 - Fecha de cada sesión.
 - Tipo de ejercicio realizado.
 - Duración de la sesión.
 - Cálculo del tiempo total: El programa debe calcular y mostrar la duración total de todas las sesiones de entrenamiento registradas. Así como el tiempo total por tipo de ejercicio realizado



3. Escribe un programa que permita a los usuarios ingresar y administrar un listado de especies de árboles. El programa debe cumplir con los siguientes requisitos:
- Registro de especies de árboles: Permitir a los usuarios ingresar el nombre de una especie de árbol. Si la especie ya está registrada, se debe lanzar una excepción personalizada (`EspecieExistenteException`) que indique que la especie ya existe en la lista.
 - Almacenamiento de datos: Los nombres de las especies de árboles deben almacenarse en una lista.
 - Reporte de especies: Permitir a los usuarios ver un resumen de todas las especies de árboles registradas.
 - Excepción personalizada: Debe crearse una excepción personalizada (`EspecieExistenteException`) que se lance cuando se intente registrar una especie que ya existe en la lista.
 - Manejo de excepciones: El programa debe manejar la excepción personalizada y mostrar un mensaje adecuado al usuario cuando se intente registrar una especie que ya existe.

El programa debería ser simple de usar y permitir a los usuarios agregar nuevas especies de árboles, ver la lista de especies registradas y manejar las excepciones correctamente.



4. Escribe un programa que permita a los usuarios votar por su película favorita de una lista predeterminada de películas. El programa debe cumplir con los siguientes requisitos:
- Lista de películas: Crea una lista predeterminada de películas (por ejemplo, ["Inception", "The Matrix", "Pulp Fiction", "The Godfather"]).
 - Votación: Permite a los usuarios ingresar su voto para una de las películas de la lista. Los usuarios deben poder ingresar el número correspondiente a la película que desean votar.
 - Registro de votos: Almacena los votos de los usuarios en un diccionario, donde la clave es el nombre de la película y el valor es el número de votos.
 - Cálculo de la película ganadora: Al finalizar el programa, muestra el nombre de la película con la mayor cantidad de votos como la ganadora.
 - Manejo de entradas no válidas: El programa debe manejar entradas no válidas de los usuarios (por ejemplo, si un usuario ingresa un número fuera del rango de la lista de películas o una opción que no es válida).

El programa debe ser sencillo de usar y permitir a los usuarios votar fácilmente por su película favorita. Además, el manejo de entradas no válidas garantizará que el programa sea robusto.



Bono 10 puntos extra:

Escribe un programa en Python que funcione como una calculadora de propinas. El programa debe realizar lo siguiente:

- Solicitar al usuario que ingrese el total de la factura.
- Presentar un menú con diferentes opciones de porcentajes de propina (por ejemplo, 10%, 15%, 20%).
- Permitir al usuario seleccionar una opción del menú.
- Calcular la cantidad de propina basada en el total de la factura y el porcentaje seleccionado.
- Mostrar el total de la factura más la propina calculada.
- El programa debe proporcionar una interfaz de usuario clara y guiar al usuario a través del proceso de selección de la opción de propina. Se espera que el código sea claro y fácil de entender, con comentarios adecuados para explicar su funcionamiento.

Los estudiantes pueden agregar características adicionales, como la capacidad de dividir la factura entre un número específico de personas, o la personalización de porcentajes de propina

Criterios de evaluación:

- **Comprensión del problema (25%):** Habilidad para analizar y comprender el problema presentado.
- **Diseño y planificación (25%):** Desarrollo de una estrategia clara y eficiente para resolver el problema.
- **Implementación del código (25%):** Escritura de un código claro, legible y bien estructurado en Python.
- **Correctitud de la solución (25%):** Verificación de que el programa produce resultados correctos según los requisitos del problema.